RankAlign Training Dynamics Analysis

When Does RankAlign / TC Improve?

Automated Analysis Report

June 9, 2026

Abstract

This report analyzes training dynamics for RankAlign models across four model/task combinations (gemma-2-9b-it and Qwen3.5-9B on membership and IFEval tasks). We characterize **(a)** when RankAlign improves correlation over the base model, **(b)** when the full method improves over basic RankAlign, and **(c)** when typicality correction (TC) specifically improves over non-TC training. We also report training diagnostics from WandB logs and additional statistics (concordance, score delta distributions).

Contents

1 Q1: When Does Each Component Help?

1.1 Main Comparison: Gen ROC AUC (TC, Self-Scoring)

Table 1 shows the generator ROC AUC (typicality-corrected) for the final model (epoch 2) evaluated on the training set with self-TC scoring.

Table 1: Generator ROC AUC (TC, self-scoring) at epoch 2. Bold = best per row. Δ relative to base.

Model / Task	Base	s1 (SFT)	s2 (RA basic)	s3 (RA full)	s4 (TC-self)
Gemma / Membership	0.851	—	0.941	0.948	**0.976**
Gemma / IFEval	0.670	0.586	0.482	0.753	**0.801**
Qwen / Membership	0.799	0.902	0.880	0.899	**0.969**
Qwen / IFEval	0.606	0.599	0.558	0.711	**0.743**

1.2 Key Findings

1. **s4 (TC-self) is the best setting in every single combination.**
2. **Basic RankAlign (s2) hurts on IFEval** — Gemma IFEval drops from 0.670 to 0.482 (below chance). Without NLL constraints, generator scores explode.
3. **The full method (s3) consistently helps**, adding +0.08–0.11 over base across all combos.
4. **TC-self (s4) adds +0.03–0.07 over s3** — a consistent, meaningful improvement from typicality-aware training.

1.3 Visualizations

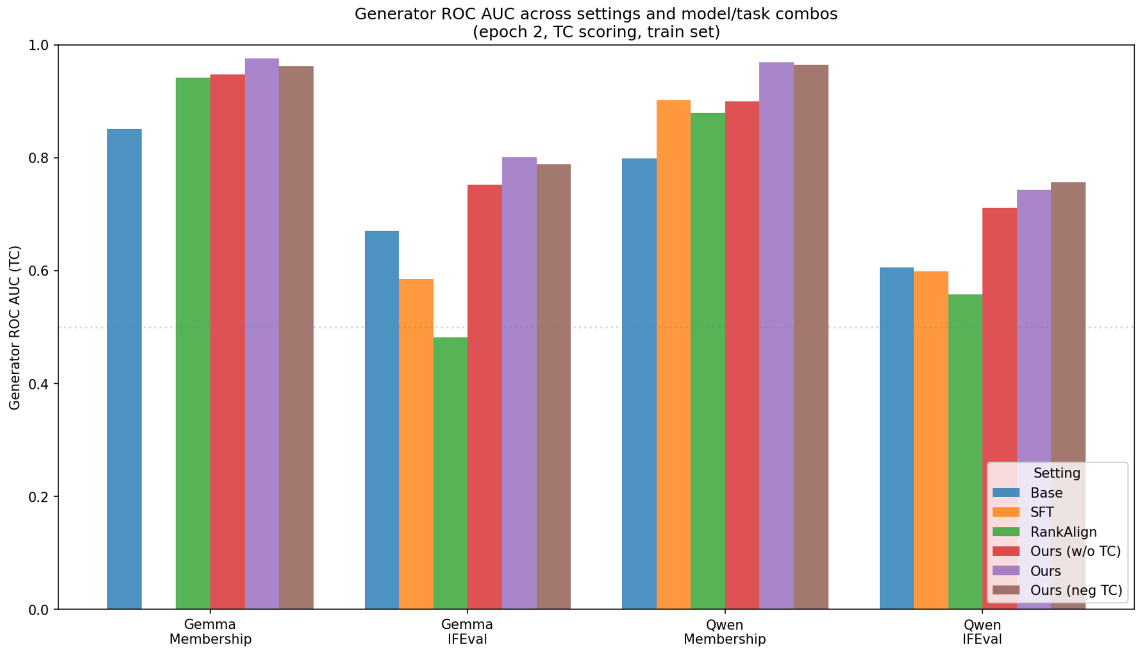

Figure 1: Generator ROC AUC across settings and model/task combinations (epoch 2, train set). Self-TC scoring is used for all settings except s7 (Ours, neg TC), which is scored with neg-TC at eval to match its training-time TC type.

2 Q2: Training Diagnostics

2.1 Loss Curves from WandB

Training was monitored via WandB for gemma-ifeval, qwen-membership, and qwen-ifeval. Gemma-membership was trained before WandB logging was added.

Smoothing. All loss curves in this section are smoothed using a running average with a window of 20 steps. To replicate in WandB: open a line plot panel's settings → Data tab → Smoothing Type: "Running Average" → Smoothing Parameter: 20. Raw (unsmoothed) curves are available directly on WandB.

2.1.1 IFEval

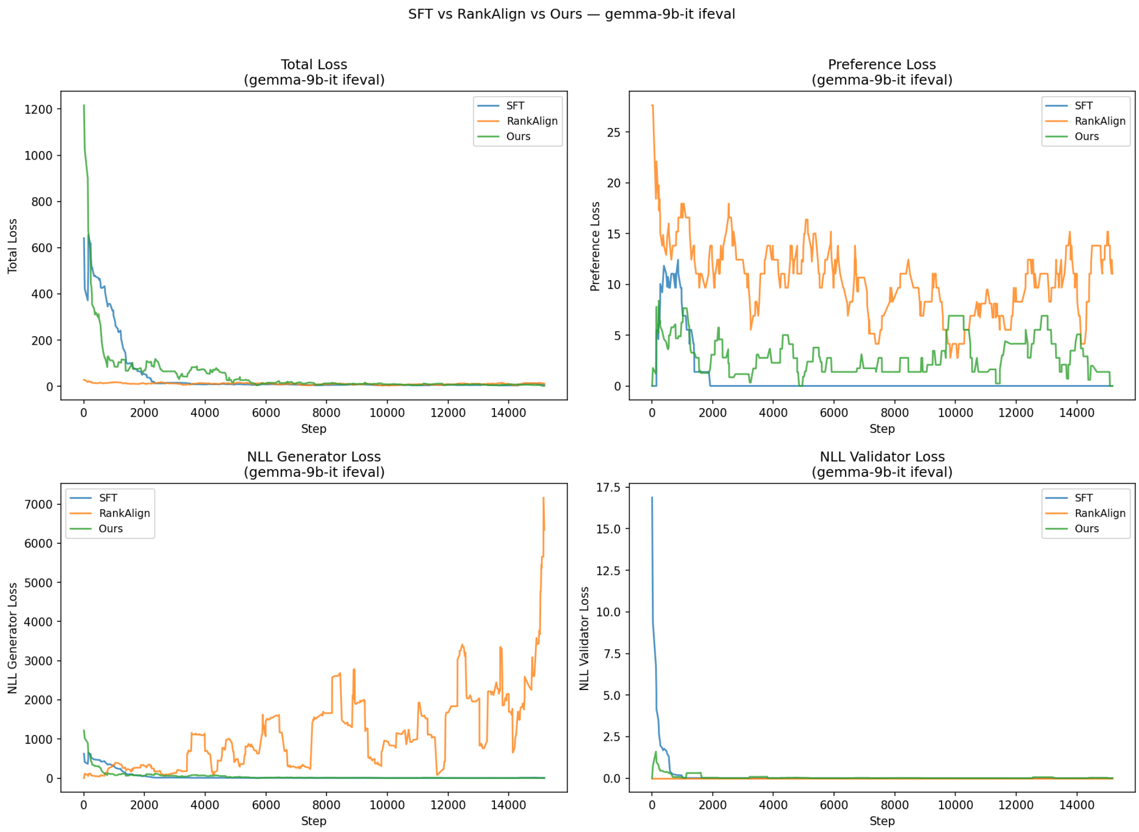

Figure 2: Loss curves: SFT vs RankAlign vs Ours (gemma-2-9b-it, IFEval).

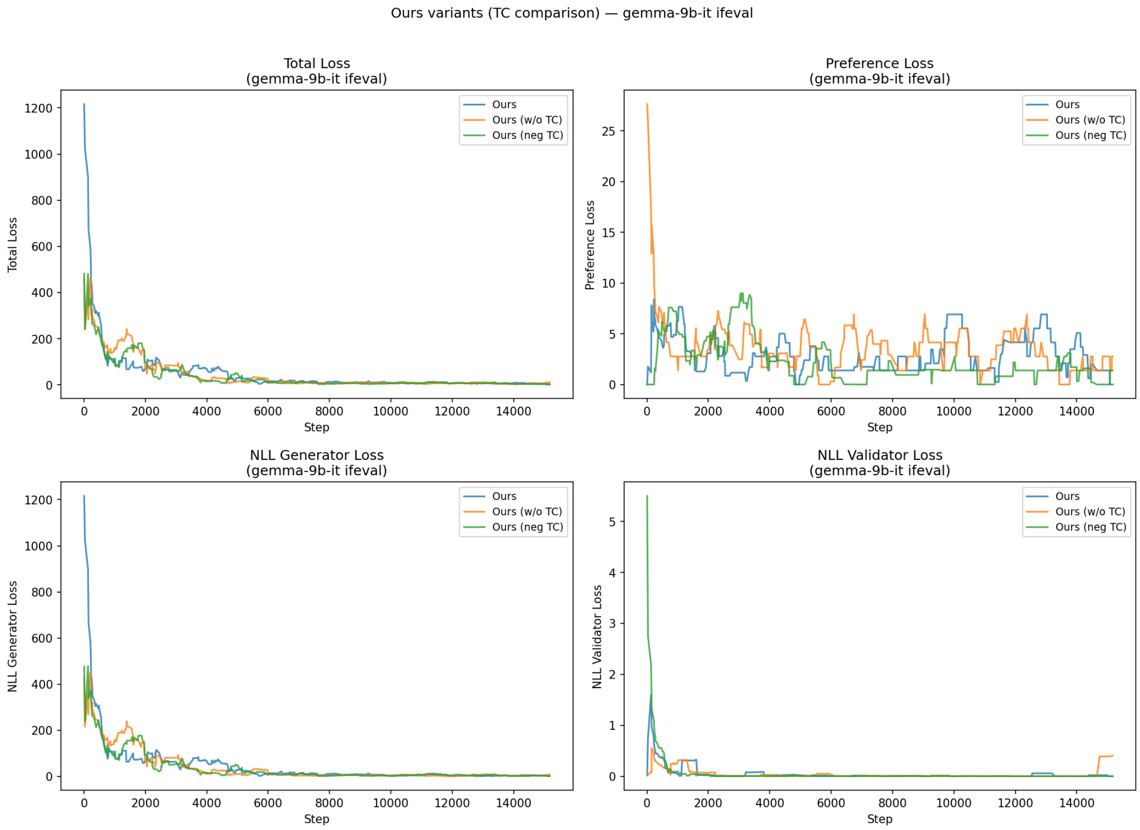

Figure 3: Loss curves: Ours TC comparison (gemma-2-9b-it, IFEval).

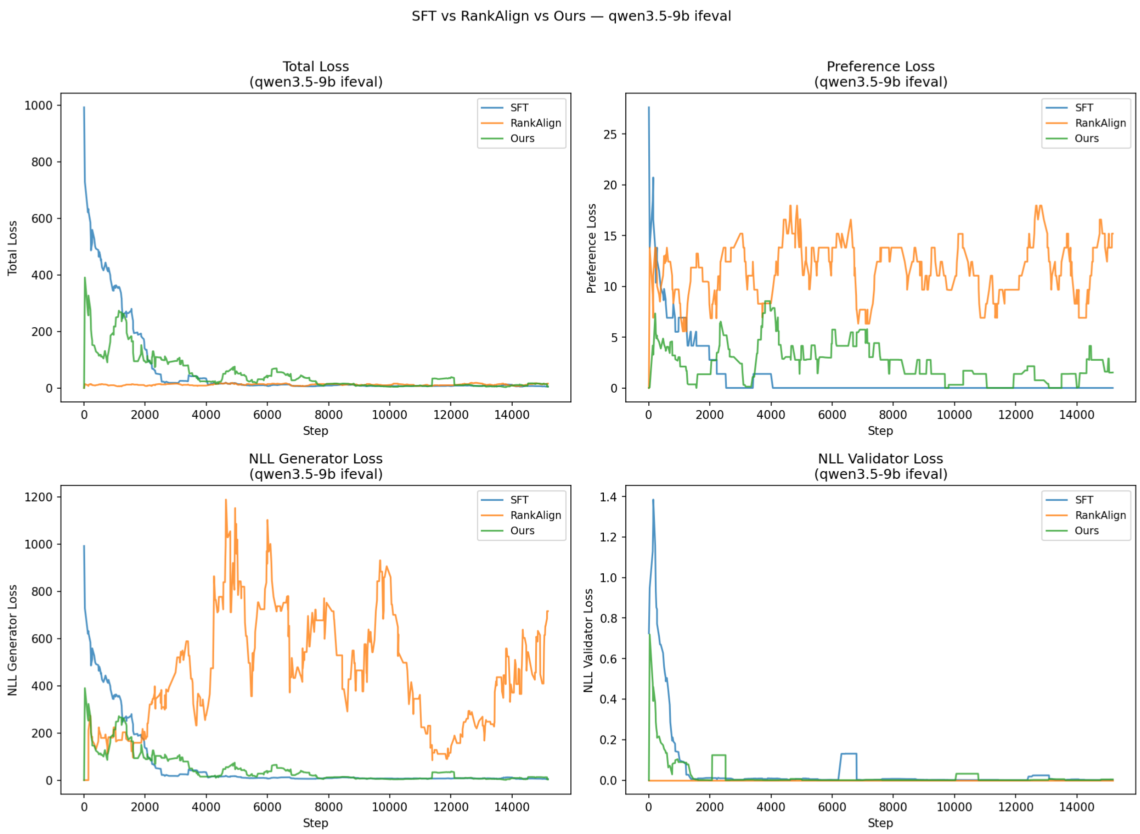

Figure 4: Loss curves: SFT vs RankAlign vs Ours (Qwen3.5-9B, IFEval).

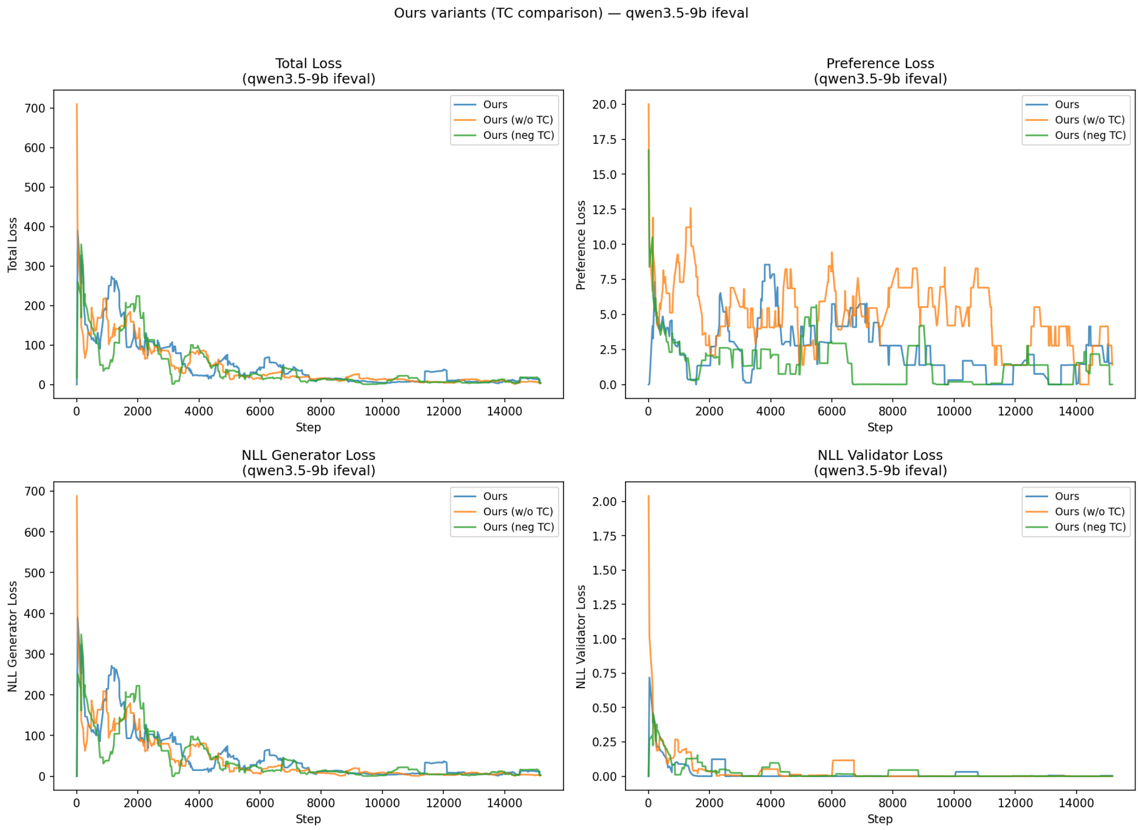

Figure 5: Loss curves: Ours TC comparison (Qwen3.5-9B, IFEval).

2.1.2 Membership

Note: gemma-2-9b-it membership was trained before WandB logging was added; no loss curves are available for that combination.

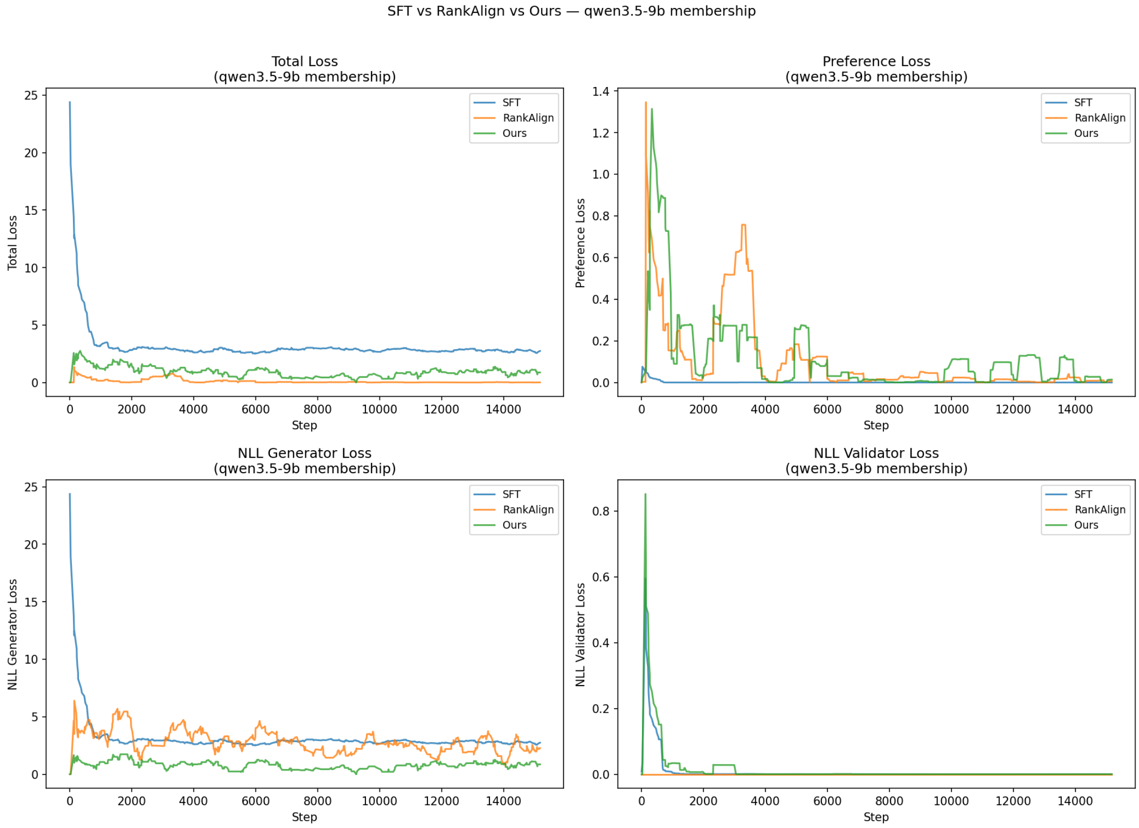

Figure 6: Loss curves: SFT vs RankAlign vs Ours (Qwen3.5-9B, membership).

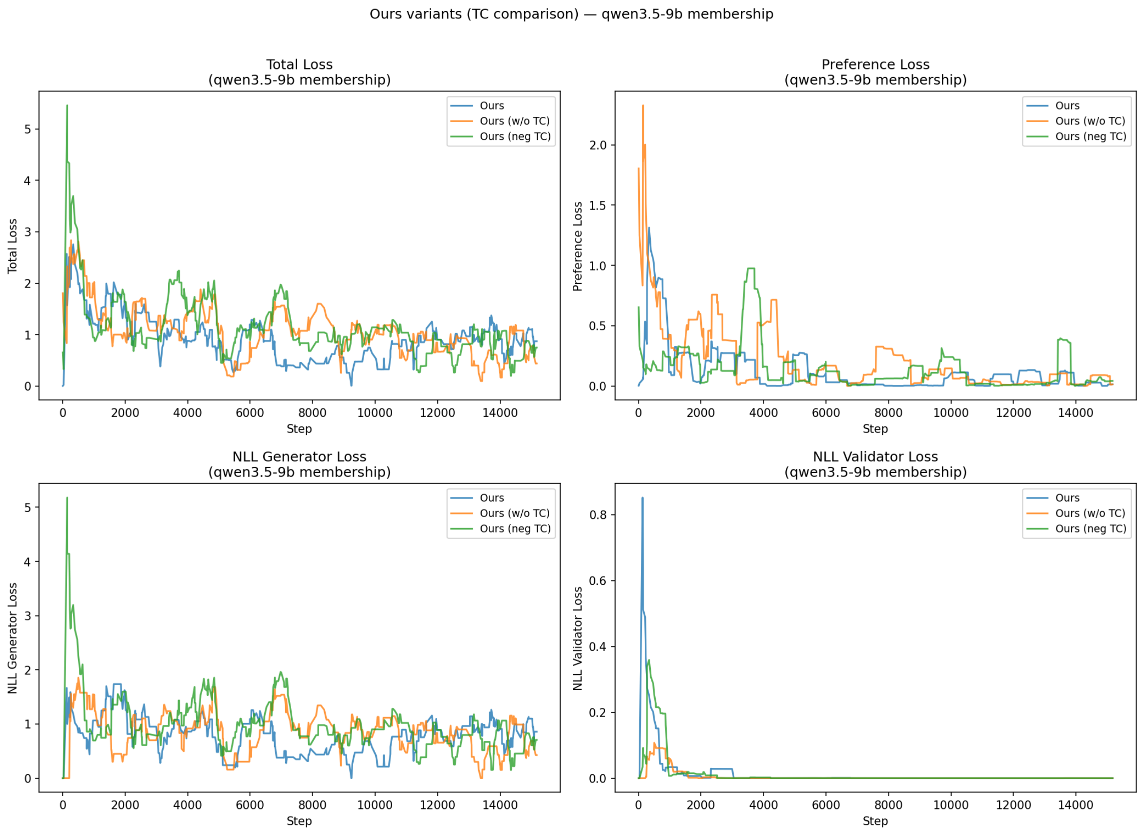

Figure 7: Loss curves: Ours TC comparison (Qwen3.5-9B, membership).

2.2 Loss Component Breakdown

Table 2 reveals the root cause of s2's failure on IFEval: without an NLL-G constraint, the generator's NLL explodes to **3120** (vs. 3.8–4.3 for constrained settings).

Table 2: Final-third loss components (Gemma IFEval). s2's NLL-G explosion explains its poor gen_roc.

Setting	Total	Pref	NLL-G	NLL-V
s1 (SFT)	5.99	0.00	5.99	0.000
s2 (basic)	**11.56**	**11.56**	**3119.8**	0.000
s4 (TC-self)	6.40	2.64	3.76	0.001
s3 (full)	6.15	1.81	4.33	0.009
s7 (TC-neg)	5.13	1.37	3.77	0.001

Key insight: On the simpler membership task (Qwen), s2 keeps NLL-G reasonable (2.42) without explicit constraint. The constraint is only critical for complex tasks where the optimization landscape allows score explosion.

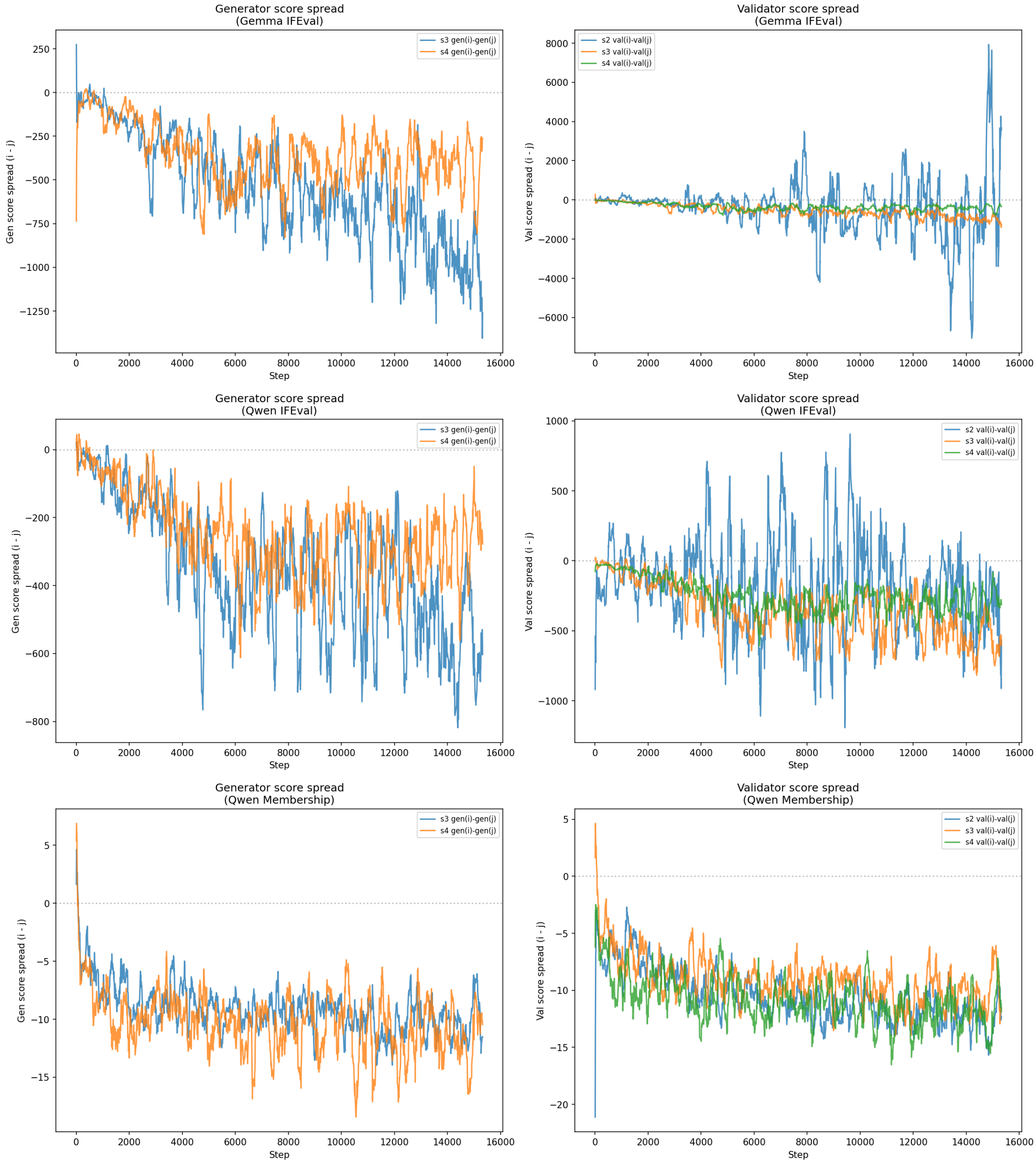

Figure 8: Score spread evolution during training. Each curve plots the difference between the two scores in a sampled training pair (item i minus item j) at each logged step, smoothed with a 20-step running average. Generator panels (left column) include only settings with $\mathrm{NLL}_G > 0$ (s2 with $\mathrm{NLL}_G = 0$ does not log per-item generator scores), so only s3 and s4 appear there; validator panels (right column) include s2, s3, s4. A more negative spread indicates a wider learned margin between the two paired items.

3 Q3: Additional Statistics

3.1 Generator-Validator Concordance

Concordance measures the fraction of item pairs where the generator and validator agree on the ordering direction.

Table 3: Concordance at epoch 2 (self-TC scoring). Chance = 0.5.

Model / Task	Base	s1	s2	s3	s4
Gemma / Membership	0.705	—	0.794	0.787	**0.803**
Gemma / IFEval	0.548	0.561	0.370	0.677	**0.694**
Qwen / Membership	0.671	0.734	0.759	0.752	**0.807**
Qwen / IFEval	0.545	0.553	0.571	0.641	**0.646**

Note: s2 on Gemma IFEval has concordance 0.370 (below chance), confirming the generator scores are *anti-correlated* with the validator.

3.2 Spearman Correlation (Generator vs Validator)

Table 4: Spearman ρ between generator and validator scores (epoch 2, self-TC).

Model / Task	Base	s1	s2	s3	s4
Gemma / Membership	0.585	—	0.798	0.795	**0.826**
Gemma / IFEval	0.141	0.178	−0.374	0.504	**0.567**
Qwen / Membership	0.496	0.665	0.723	0.715	**0.830**
Qwen / IFEval	0.119	0.151	0.205	0.404	**0.431**

3.3 Pearson Correlation (Generator vs Validator)

Table 5: Pearson r between generator and validator scores (epoch 2, self-TC).

Model / Task	Base	s1	s2	s3	s4
Gemma / Membership	0.563	—	0.777	0.798	**0.863**
Gemma / IFEval	0.180	0.167	−0.313	**0.551**	0.483
Qwen / Membership	0.515	0.666	0.709	0.702	**0.818**
Qwen / IFEval	0.100	0.168	0.139	**0.472**	0.351

Pearson tracks Spearman on the membership tasks (s4 highest). On the IFEval tasks, s3 has higher Pearson than s4 even though s4 has higher Spearman — s4 preserves rank order while compressing absolute magnitudes via TC.

3.4 Score Delta Distributions

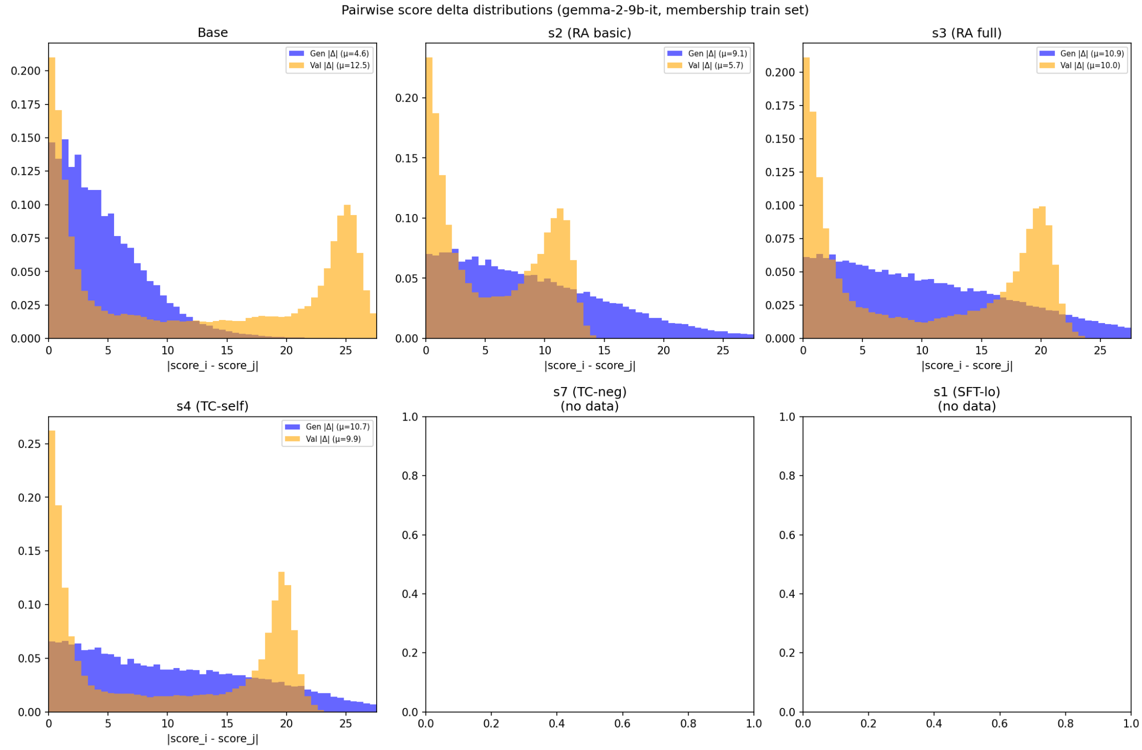

Figure 9: Pairwise score delta distributions for gemma membership. Generator (blue) and validator (orange) distributions. Tighter = better calibrated.

3.5 Training Dynamics

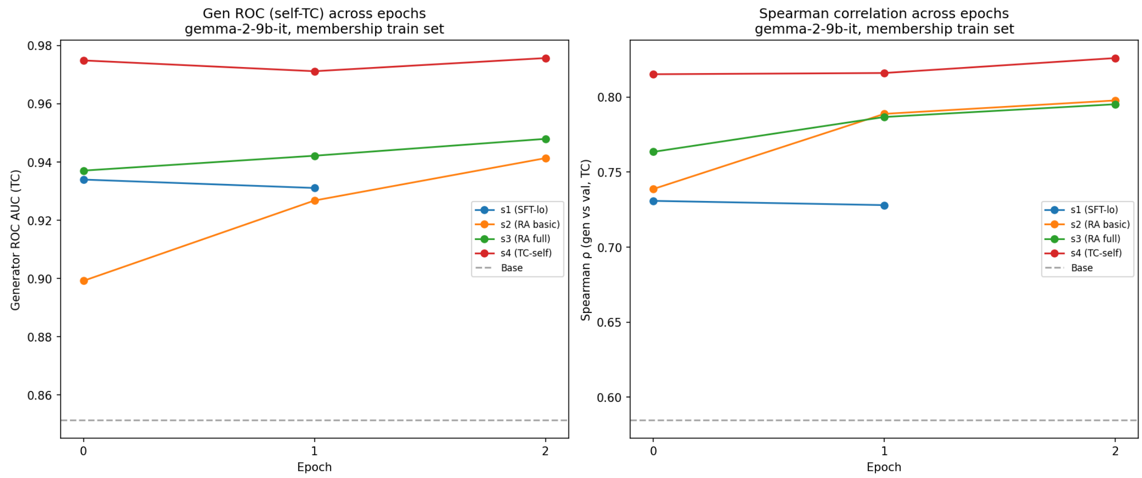

Figure 10: Training dynamics for gemma membership: Gen ROC and Spearman across epochs.

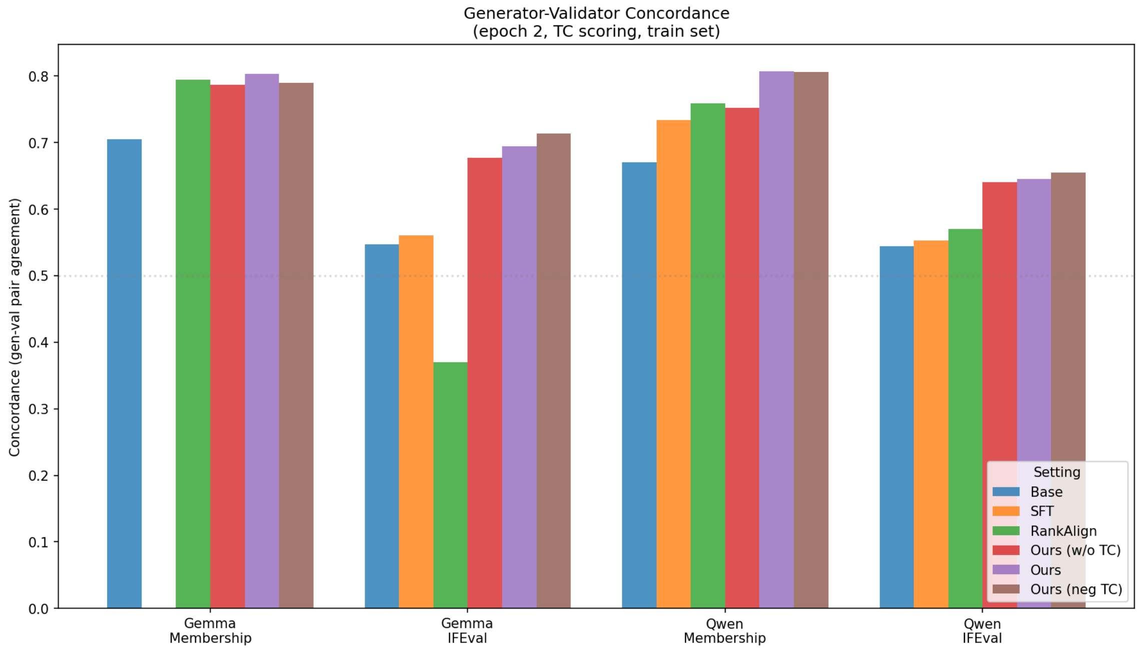

Figure 11: Concordance across all model/task combinations.

3.6 Generator-Validator Scatter Plots

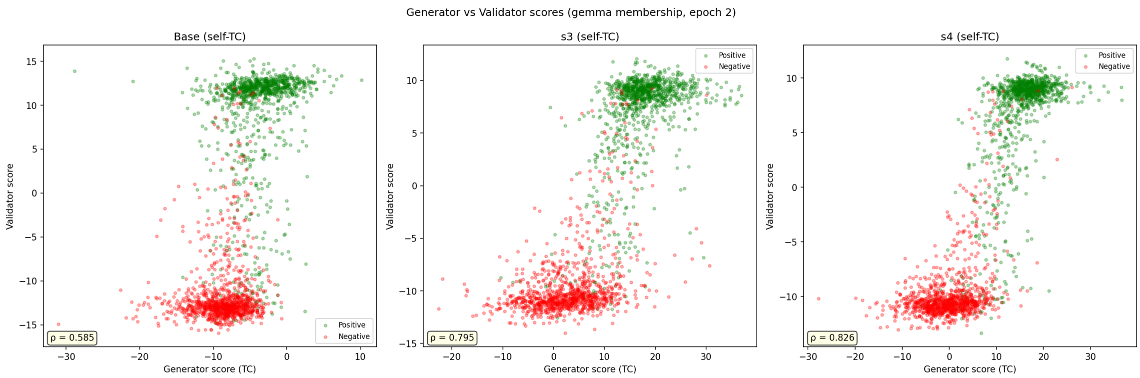

Figure 12: Generator vs validator scores for Base, s3, and s4 (self-TC scoring, gemma membership). Training improves the separation between positive (green) and negative (red) items in both generator and validator space.

3.7 Per-Category Analysis

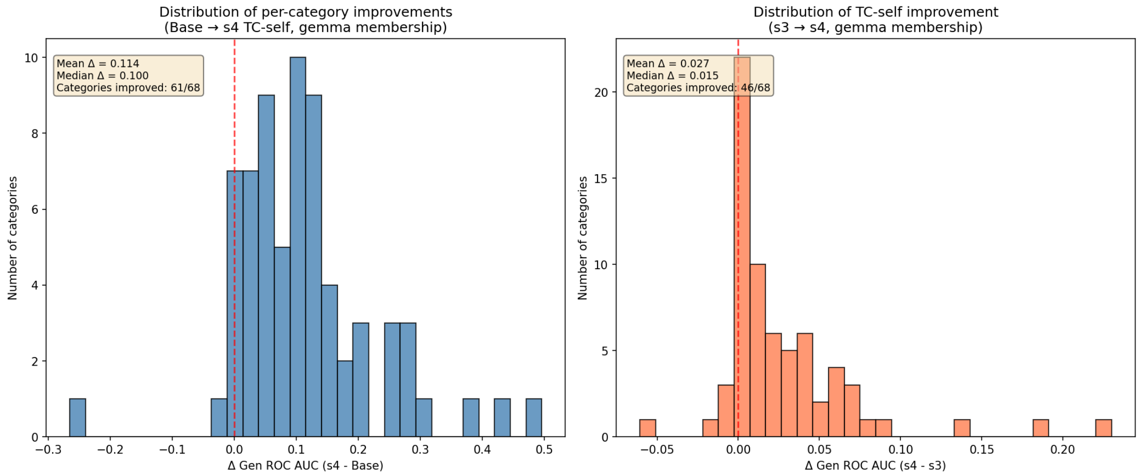

Figure 13: Left: distribution of per-category improvements (Base → s4). Right: improvements from TC-self specifically (s3 → s4). Both are consistently positive.

Categories where the base model was weakest (gen_roc ≈ 0.5–0.7) benefit most from training. For example, "medical specialty" improves from 0.504 to 1.000 (+0.496). Categories already near ceiling show minimal gains (expected).

4 Focused TC Comparison

4.1 Self-TC vs No-TC (s4 vs s3)

When evaluated with self-TC scoring, the typicality-corrected models consistently outperform the non-TC baseline (s3):

- s4 (TC-self, full): +0.028 over s3
- s6 (TC-self fsx, low δ): +0.023 over s3
- s11 (TC-self vlo, no fsx/ppd): +0.022 over s3
- s5 (TC-self basic): +0.010 over s3

4.2 Neg-TC vs No-TC (s7 vs s3)

When evaluated with neg-TC scoring, the improvement from TC-neg training is even larger:

- s12 (TC-neg vlo): +0.075 over s3
- s7 (TC-neg, full): +0.072 over s3

This suggests that TC-trained models learn better internal representations that are specifically captured by the matching TC scoring method.

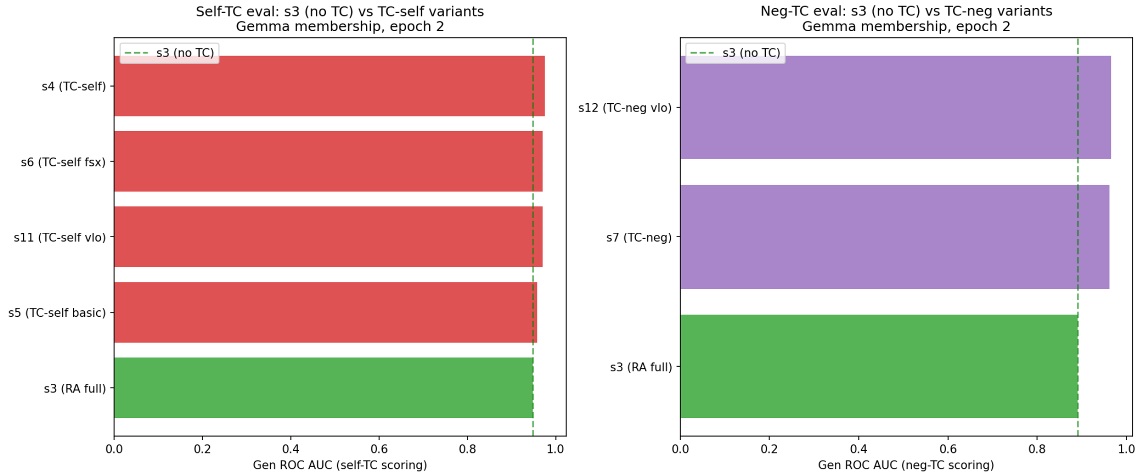

Figure 14: TC comparison: self-TC variants (left) and neg-TC variants (right) against s3 baseline.

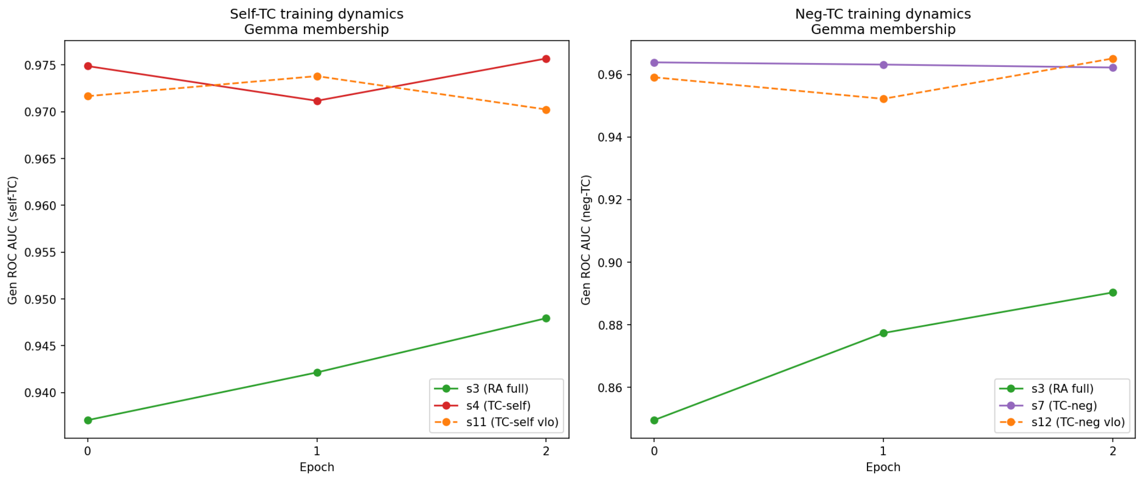

Figure 15: Epoch-by-epoch dynamics for TC comparison. TC-trained models converge quickly and maintain advantage throughout training.

5 Train vs Test Generalization

A critical finding emerges when comparing train-set dynamics with held-out test sets.

Table 6: Train vs test gen_roc: gemma-2-9b-it membership/rosch (basetyp scoring).

Setting	**Train (membership)**	**Test (rosch)**	
s11 (TC-self vlo)	0.970	**0.924**	−0.046
s4 (TC-self full)	**0.976**	0.920	−0.056
s5 (TC-self basic)	0.958	0.913	−0.045
s2 (RA basic)	0.941	0.891	−0.050
s3 (RA full)	0.948	0.885	−0.063

Key finding: s3 (full method, no TC) outperforms s2 on training categories but *underperforms* on test categories (different Rosch categories). Meanwhile, all TC-trained models (s4, s11, s5) maintain their advantage on both splits. This suggests **TC promotes generalization** — it prevents overfitting to training-specific patterns.

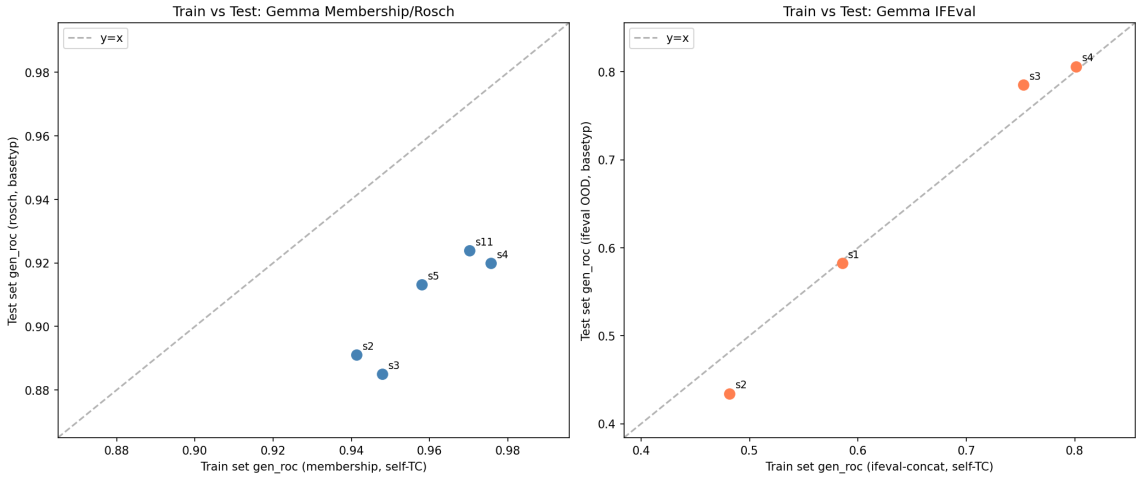

Figure 16: Train vs test set gen_roc. Left: membership train ↔ rosch test. Right: ifeval train ↔ ifeval OOD test.

6 Conclusions

1. **RankAlign works when the full method (s3) is used** — the NLL-V/G constraints are essential for preventing score explosion, especially on complex tasks like IFEval.
2. **TC-self training (s4) consistently provides the best results** across all metrics (gen ROC, spearman, concordance) and all model/task combinations.
3. **TC promotes generalization**: TC-trained models maintain their advantage on held-out test categories, while non-TC models (s3) can overfit to training patterns.
4. **Basic RankAlign (s2) is unreliable for complex tasks** — it can actually *hurt* performance (IFEval). The full method is needed.
5. **Score spread (gen_delta_mean) is a powerful training diagnostic** — values >1000 indicate instability. TC-self produces the tightest distributions.
6. **Concordance is a complementary metric to ROC AUC** — it directly measures the alignment between generator and validator ranking functions.